Mês 2 — Partição e Teoria WPT (out/2026)

Pesquisa PIBITI - Detecção de Deepfake de Áudio

Outubro de 2026

Sumário

1 O QUE O PLANO EXIGE (plan vs practice) 2
1.1 Atividades Previstas 2

2 PARTIÇÃO DOS DADOS 4
2.1 O que é Train/Dev/Test e por quê? 4
2.2 Speaker-Level Split e Data Leakage 4
2.3 Criando o Arquivo de Partição 5
2.4 Validando a Partição (Evitando Overlap) 5
2.5 Balanceamento de Classes 6
2.6 Congelando a Partição 6

3 TEORIA WPT AVANÇADA 7
3.1 Nível L e Sub-bandas 7
3.2 Wavelets Mãe: db4, db8 e sym8 7
3.3 Reconstrução de Sub-bandas no Domínio do Tempo 8
3.4 Entropia de Shannon para Seleção de Base 8

4 PADRONIZAÇÃO DO ÁUDIO 10
4.1 Comparativo SoundFile vs Librosa 10
4.2 Funções do Pipeline 10

5 TRABALHANDO COM ARQUIVOS DE ÁUDIO 12
5.1 Listando Arquivos e Lotes (Batches) 12

6 BRINCADEIRAS DO MÊS 2 14

7 CHECKLIST DO MÊS 2 16

8 DESAFIO FINAL DO MÊS 2 17

Capítulo 1

O QUE O PLANO EXIGE (plan vs practice)

Neste segundo mês, os objetivos práticos e teóricos convergem para estabelecer uma base sólida de experimentação. Os modelos de *Machine Learning* dependem fortemente de dados organizados e de características (*features*) extraídas corretamente. Portanto, precisamos abordar a partição do *dataset* e dominar a matemática e o código por trás da *Wavelet Packet Transform* (WPT).

Marco / Entrega

Objetivo do Mês 2: Garantir o arquivo de partição do corpus (adotando um existente ou gerando um novo) e dominar a teoria WPT. O resultado final será o arquivo de partição validado e anotações teóricas consolidadas sobre WPT.

1.1 Atividades Previstas

- **Partição de Dados:** Se a frente de *benchmark* do laboratório já estiver operando, adotaremos o arquivo de partição versionado (congelado) para mantermos consistência, verificando sempre sua integridade. Caso esta pesquisa seja a única frente em andamento, geraremos o nosso próprio arquivo com as seguintes especificações: 16kHz, mono, anotações de classes e divisão estrita por locutor (*speaker-level split*).
- **Aprofundamento em WPT:** Compreender e codificar a extração em níveis profundos (L entre 4 e 5), analisar famílias de *wavelets*-mãe como `db4`, `db8` e `sym8`, realizar a reconstrução das sub-bandas no domínio do tempo e estudar a medida de entropia para a seleção de bases.
- **Viabilidade:** Confirmar um subconjunto balanceado cujas dimensões caibam no orçamento computacional de CPU disponível.

Atenção

O arquivo de partição é ÚNICO e deve ser CONGELADO. Nunca re-particione os dados após defini-lo, ou você invalidará qualquer comparação futura entre seus modelos experimentais!

Leituras Recomendadas

- Coifman, R. R., & Wickerhauser, M. V. (1992). *Entropy-based algorithms for best basis selection.*
- Lee et al. (2019). Documentação do pacote *PyWavelets.*
- Artigos originais do corpus adotado (ex: ASVspoof 2019, FoR, etc.).

Capítulo 2

PARTIÇÃO DOS DADOS

2.1 O que é Train/Dev/Test e por quê?

Em aprendizado de máquina, nós nunca avaliamos o modelo nos mesmos dados usados para treiná-lo. Precisamos dividi-los em:

- **Train (Treino):** Dados usados para ajustar os pesos do modelo.
- **Dev/Val (Validação):** Dados usados para ajustar hiperparâmetros (como profundidade da WPT ou taxa de aprendizado).
- **Test (Teste):** Dados usados apenas uma vez, no final, para simular o mundo real.

2.2 Speaker-Level Split e Data Leakage

Data Leakage (vazamento de dados) ocorre quando a rede neural "decora"características que não deveriam estar disponíveis. Em áudio de voz, o erro mais comum é o **vazamento de locutor** (*speaker leakage*).

Dica

Se o "Locutor A"estiver no conjunto de treino e também no de teste, o modelo pode aprender a reconhecer o "timbre do Locutor A"em vez de aprender a detectar deepfake. Quando um áudio novo chegar de um locutor desconhecido, o modelo vai falhar miseravelmente.

Abaixo, vemos uma simulação de como criar um dicionário de metadados falso:

Listing 2.1: Criando um conjunto de dados falso

```
import pandas as pd
import numpy as np

# Simulando metadados de audio
data = {
    'arquivo': [f'audio_{i}.wav' for i in range(100)],
    'locutor': [f'spk_{i % 10}' for i in range(100)], # 10 locutores
    'label': ['bonafide' if i % 2 == 0 else 'spoof' for i in range(100)]
```

```
}
df_corpus = pd.DataFrame(data)
```

2.3 Criando o Arquivo de Partição

O código a seguir isola os locutores para garantir o *speaker-level split*.

Listing 2.2: Divisão estrita por locutores

```
# 1. Obter locutores unicos
locutores = df_corpus['locutor'].unique()
np.random.seed(42) # Reprodutibilidade eh essencial!
np.random.shuffle(locutores)

# 2. Definir proporcoes (60% Treino, 20% Dev, 20% Teste)
n_treino = int(0.6 * len(locutores))
n_dev = int(0.2 * len(locutores))

spk_train = locutores[:n_treino]
spk_dev = locutores[n_treino:n_treino+n_dev]
spk_test = locutores[n_treino+n_dev:]
```

Agora, mapeamos de volta para o dataframe:

Listing 2.3: Atribuindo partições aos arquivos

```
def assign_partition(spk):
    if spk in spk_train: return 'train'
    elif spk in spk_dev: return 'dev'
    else: return 'test'

df_corpus['partition'] = df_corpus['locutor'].apply(
   assign_partition)
print(df_corpus['partition'].value_counts())
```

2.4 Validando a Partição (Evitando Overlap)

Listing 2.4: Validando vazamento de locutor

```
def check_leakage(df):
    train_spks = set(df[df['partition'] == 'train']['locutor'])
    dev_spks = set(df[df['partition'] == 'dev']['locutor'])
    test_spks = set(df[df['partition'] == 'test']['locutor'])

    # Interseccao deve ser vazia
    leak_dev = train_spks.intersection(dev_spks)
    leak_test = train_spks.intersection(test_spks)

    if leak_dev or leak_test:
        raise ValueError("VAZAMENTO DE LOCUTOR DETECTADO!")
    print("Sucesso! Nenhum vazamento de locutor encontrado.")
```

```
13
14 check_leakage(df_corpus)
```

2.5 Balanceamento de Classes

Uma base onde 90% dos dados é deepfake fará o modelo chutar "deepfake"sempre. Precisamos balancear.

Listing 2.5: Balanceando classes do subconjunto

```
1  def balance_classes(df, part='train'):
2      df_part = df[df['partition'] == part]
3      min_count = df_part['label'].value_counts().min()
4
5      df_bonafide = df_part[df_part['label'] == 'bonafide'].sample(
           min_count, random_state=42)
6      df_spoof = df_part[df_part['label'] == 'spoof'].sample(
           min_count, random_state=42)
7
8      return pd.concat([df_bonafide, df_spoof])
9
10 df_train_bal = balance_classes(df_corpus, 'train')
11 print("Treino balanceado:\n", df_train_bal['label'].value_counts())
```

2.6 Congelando a Partição

Listing 2.6: Salvando o arquivo congelado

```
1 # Vamos salvar o dataframe inteiro. NUNCA MAIS SOBREESCREVA!
2 df_corpus.to_csv('particao_frozen_v1.csv', index=False)
```

Exercício

Crie um script em Python que leia `particao_frozen_v1.csv`, remova 5 arquivos do conjunto de dev, salve como outro nome, e compare os hashes SHA-256 dos dois arquivos para provar que a modificação corrompeu o "congelamento".

Capítulo 3

TEORIA WPT AVANÇADA

3.1 Nível L e Sub-bandas

Ao contrário da Transformada Wavelet Discreta (DWT), que decompõe apenas as aproximações de baixa frequência, a WPT (Wavelet Packet Transform) decompõe tanto as aproximações (baixa) quanto os detalhes (alta), gerando uma árvore completa binária.

No nível L, a quantidade de sub-bandas geradas é 2^L.

Nível (L)	**Número de Sub-bandas (2^L)**	**Complexidade da Árvore**
1	2	Raiz → [a, d]
2	4	[aa, ad, da, dd]
3	8	8 nós terminais
4	16	16 nós terminais (usado em áudio)
5	32	32 nós terminais (ótimo para 16kHz)
6	64	Computacionalmente custoso

Tabela 3.1: Relação entre nível e sub-bandas na WPT.

3.2 Wavelets Mãe: db4, db8 e sym8

As wavelets mãe possuem diferentes números de momentos de fuga (vanishing moments) e suporte.

Listing 3.1: Analisando famílias PyWavelets

```
import pywt

print("Daubechies:", pywt.wavelist(family='db')[:10])
print("Symlets:", pywt.wavelist(family='sym')[:10])
```

Listing 3.2: Extraindo WPT com db4

```
import numpy as np
# Simulando sinal de audio de 1 segundo a 16kHz
sinal = np.random.randn(16000)

```

```
wp_db4 = pywt.WaveletPacket(data=sinal, wavelet='db4', mode='
    symmetric', maxlevel=5)
```

Listing 3.3: Inspecionando os nós da árvore

```
# Obtendo o caminho de um no no nivel 5 (ex: 'aaaaa' ou 'adada')
no_exemplo = wp_db4['aaaaa']
print(f"Nome do no: {no_exemplo.path}")
print(f"Tamanho dos coeficientes: {len(no_exemplo.data)}")
```

3.3 Reconstrução de Sub-bandas no Domínio do Tempo

Os coeficientes de um nó representam aquele intervalo de frequência. Para voltar ao domínio do tempo, devemos zerar os outros nós e realizar a reconstrução inversa (IWPT).

Listing 3.4: Reconstruindo uma sub-banda

```
wp_vazio = pywt.WaveletPacket(data=np.zeros_like(sinal), wavelet='
    db4', mode='symmetric')

# Copia os coeficientes de 'aaaaa' (baixas frequencias) para a
    arvore vazia
wp_vazio['aaaaa'] = wp_db4['aaaaa'].data

# Reconstroi o sinal no tempo usando apenas essa sub-banda
sinal_filtrado = wp_vazio.reconstruct(update=False)
print("Tamanho do sinal reconstruido:", len(sinal_filtrado))
```

3.4 Entropia de Shannon para Seleção de Base

A árvore completa WPT tem muitos nós redundantes. Coifman & Wickerhauser (1992) propuseram usar uma métrica de entropia para decidir se um nó pai deve ser dividido nos seus dois filhos.

Listing 3.5: Calculando a Entropia de Shannon

```
def shannon_entropy(data):
    # Calcula a energia e normaliza
    energia = data ** 2
    soma_energia = np.sum(energia)

    if soma_energia == 0:
        return 0

    p = energia / soma_energia
    p = p[p > 0] # Evita log(0)

    # Formula da entropia
    return -np.sum(p * np.log(p))

entropia_pai = shannon_entropy(wp_db4['a'].data)
```

```
print(f"Entropia do pai 'a': {entropia_pai:.4f}")
```

Listing 3.6: Lógica de Coifman & Wickerhauser

```
# O no pai so eh desmembrado se a soma da entropia dos filhos for
    MENOR
entropia_filho1 = shannon_entropy(wp_db4['aa'].data)
entropia_filho2 = shannon_entropy(wp_db4['ad'].data)
entropia_filhos_total = entropia_filho1 + entropia_filho2

if entropia_filhos_total < entropia_pai:
    print("Melhor dividir! Filhos tem menos entropia conjunta.")
else:
    print("Manter o pai! Dividir nao compensa.")
```

Listing 3.7: Comparando Wavelets Diferentes

```
wp_db8 = pywt.WaveletPacket(data=sinal, wavelet='db8', maxlevel=5)
wp_sym8 = pywt.WaveletPacket(data=sinal, wavelet='sym8', maxlevel
    =5)

print("DB8 Energia Total:", np.sum(wp_db8['aaaaa'].data ** 2))
print("SYM8 Energia Total:", np.sum(wp_sym8['aaaaa'].data ** 2))
```

Capítulo 4

PADRONIZAÇÃO DO ÁUDIO

Todo deepfake deve ser avaliado no mesmo formato. Um modelo treinado com áudio de 44.1kHz estéreo quebrará ao receber 16kHz mono. O pipeline padrão exigido é: `Ler` → `Mono` → `16kHz`.

4.1 Comparativo SoundFile vs Librosa

A biblioteca `librosa` carrega o áudio, converte para mono e reamostra (tudo a 22050Hz por padrão, cuidado!). `soundfile` carrega os dados brutos e costuma ser mais rápida, porém não faz reamostragem nativa.

Listing 4.1: Lendo dados com Librosa vs SoundFile

```
import librosa
import soundfile as sf
import os

# Simularemos gravando um arquivo mudo
sf.write('teste_dummy.wav', np.zeros(44100), 44100)

# Metodo Librosa (Forcando nao reamostrar)
audio_lib, sr_lib = librosa.load('teste_dummy.wav', sr=None, mono=
    False)

# Metodo SoundFile
audio_sf, sr_sf = sf.read('teste_dummy.wav')
```

4.2 Funções do Pipeline

Listing 4.2: Função: Conversão para Mono

```
def converter_mono(audio_array):
    if len(audio_array.shape) > 1:
        # Faz a media dos dois canais (est reo)
        return np.mean(audio_array, axis=1)
    return audio_array
```

Listing 4.3: Função: Reamostragem (Resampling)

```python
def reamostrar(audio_array, sr_atual, sr_desejado=16000):
    if sr_atual != sr_desejado:
        # Usando librosa para reamostrar arrays numpy
        return librosa.resample(y=audio_array, orig_sr=sr_atual,
            target_sr=sr_desejado)
    return audio_array
```

Listing 4.4: Pipeline Completo Integrado

```python
def pipeline_audio(caminho_arquivo):
    # 1. Carregar usando soundfile (mais rapido)
    audio, sr = sf.read(caminho_arquivo)

    # 2. Garantir mono
    audio = converter_mono(audio)

    # 3. Garantir 16kHz
    audio = reamostrar(audio, sr, 16000)

    return audio, 16000
```

Exercício

Crie um arquivo WAV de 2 canais (estéreo) a 48kHz. Passe ele pela sua função `pipeline_audio`. Verifique com `audio.shape` se a saída ficou unidimensional (mono) e registre quanto tempo a etapa de `reamostrar` demorou usando o módulo `time`.

Capítulo 5

TRABALHANDO COM ARQUIVOS DE ÁUDIO

Para processar milhares de arquivos do dataset LA (ASVspoof) ou FoR, não podemos fazer as coisas manualmente.

5.1 Listando Arquivos e Lotes (Batches)

Listing 5.1: Lendo metadados rapidamente

```
def info_audio(caminho):
    info = sf.info(caminho)
    return {
        'sr': info.samplerate,
        'canais': info.channels,
        'duracao_s': info.duration,
        'amostras': info.frames
    }

print(info_audio('teste_dummy.wav'))
```

Listing 5.2: Uso da biblioteca glob para batches

```
import glob

# Encontra recursivamente todos os wavs em subpastas
arquivos = glob.glob('meu_dataset/**/*.wav', recursive=True)
print(f"Total de audios encontrados: {len(arquivos)}")
```

Listing 5.3: Uso de pathlib para batches

```
from pathlib import Path

# Pathlib e mais moderno e robusto
pasta = Path('.')
arquivos_wav = list(pasta.rglob('*.wav'))
for arq in arquivos_wav[:3]:
    print(arq.name, arq.stat().st_size, "bytes")
```

Listing 5.4: Processamento em Lote (Batch)

```
def processar_lote(lista_caminhos, pasta_saida):
    os.makedirs(pasta_saida, exist_ok=True)

    for caminho in lista_caminhos:
        nome_arq = os.path.basename(caminho)
        saida_caminho = os.path.join(pasta_saida, nome_arq)

        # Pipeline padrao (Carrega, Mono, 16kHz)
        aud, sr = pipeline_audio(caminho)

        # Salva o arquivo pre-processado
        sf.write(saida_caminho, aud, sr)
```

Capítulo 6

BRINCADEIRAS DO MÊS 2

Nesta seção, realizaremos 8 pequenas simulações (*toy scripts*) em Python para fixar o conteúdo.

Listing 6.1: Exp 1: Gerar e salvar uma onda senoidal (Tom puro)

```
import numpy as np
import soundfile as sf

fs = 16000
t = np.linspace(0, 1, fs, endpoint=False) # 1 segundo
freq = 440.0 # La (A4)
onda = 0.5 * np.sin(2 * np.pi * freq * t)

sf.write('senoidal_440.wav', onda, fs)
```

Listing 6.2: Exp 2: Adicionar ruído branco e visualizar

```
ruido = np.random.normal(0, 0.05, len(onda))
onda_ruidosa = onda + ruido
sf.write('senoidal_ruido.wav', onda_ruidosa, fs)
print("Relacao Sinal-Ruido adulterada.")
```

Listing 6.3: Exp 3: Simulador de Leakage de Locutor

```
# Simulando a extracao de features para 2 locutores diferentes
spk1_features = np.random.rand(10, 5) # 10 audios, 5 features
spk2_features = np.random.rand(10, 5) + 2.0 # Spk2 tem features
    deslocadas

# Se misturarmos SPK1 no treino e teste, o modelo so aprende o "+
    2.0"
train_x = np.vstack([spk1_features[:8], spk2_features[:8]])
test_x = np.vstack([spk1_features[8:], spk2_features[8:]])
print("Data Leakage instanciado. O teste vai falhar no mundo real!"
    )
```

Listing 6.4: Exp 4: Timer comparativo de extração WPT

```
import time
t0 = time.time()
```

```
wp_test = pywt.WaveletPacket(data=onda_ruidosa, wavelet='db4',
    maxlevel=5)
t1 = time.time()
print(f"Extracao WPT db4: {(t1-t0)*1000:.2f} ms")
```

Listing 6.5: Exp 5: Tamanhos dos nós no Nível 5

```
nos_nivel5 = [no.path for no in wp_test.get_level(5, 'freq')]
print(f"O primeiro no tem {len(wp_test[nos_nivel5[0]].data)}
    coeficientes.")
print(f"O total de n s     {len(nos_nivel5)}.")
```

Listing 6.6: Exp 6: Entropia de silêncio vs ruído

```
silencio = np.zeros(16000)
ruido_puro = np.random.randn(16000)

print("Entropia Silencio:", shannon_entropy(silencio))
print("Entropia Ruido:", shannon_entropy(ruido_puro))
```

Listing 6.7: Exp 7: Check de CPU com dataset Dummy

```
n_arquivos = 500
t0 = time.time()
# Simulando carregar e aplicar WPT basico em 500 audios de 1s
for _ in range(n_arquivos):
    _ = pywt.WaveletPacket(np.random.randn(16000), 'db4', maxlevel
        =3)
t1 = time.time()
print(f"Tempo para processar {n_arquivos} audios dummy: {t1-t0:.2f}
    s")
```

Listing 6.8: Exp 8: Erro de Reconstrução Quadrático (MSE)

```
onda_original = onda
wp_r = pywt.WaveletPacket(onda_original, 'sym8', maxlevel=4)
onda_reconstruida = wp_r.reconstruct(update=False)

# Corta os arrays pro mesmo tamanho (artefatos de borda)
min_len = min(len(onda_original), len(onda_reconstruida))
mse = np.mean((onda_original[:min_len] - onda_reconstruida[:min_len
    ])**2)
print(f"MSE da reconstrucao sym8: {mse}")
```

Capítulo 7

CHECKLIST DO MÊS 2

Garantir que o dataset de treino está no formato 16kHz, mono.

Construir e salvar o DataFrame de metadados garantindo *Speaker-Level Split*.

Executar a rotina de validação e comprovar ausência de vazamento de dados.

Escrever código para instanciar a *Wavelet Packet Transform* usando PyWavelets.

Analisar o número de sub-bandas geradas para $L = 4$ e $L = 5$.

Realizar o cálculo de Entropia de Shannon em uma árvore instanciada.

Implementar o laço de reconstrução do sinal a partir dos coeficientes dos nós WPT.

Capítulo 8

DESAFIO FINAL DO MÊS 2

Desafio

Junte todo o conhecimento e escreva uma única classe em Python chamada `WPTFeatureExtractor`. Ela deve receber o caminho de um áudio, processá-lo com o seu `pipeline_audio`, instanciar a WPT no nível especificado, calcular a entropia de cada nó final, e retornar um *array* 1D (vetor) com esses valores de entropia, que servirá de *feature* final para a Inteligência Artificial.

Listing 8.1: Template do Desafio Final

```python
class WPTFeatureExtractor:
    def __init__(self, wavelet='db4', level=5):
        self.wavelet = wavelet
        self.level = level

    def extrair(self, caminho_audio):
        # 1. Pipeline: mono, 16kHz
        # 2. Instanciar wp
        # 3. Pegar nos do nivel (level)
        # 4. Calcular entropia de shannon de cada um
        # 5. Retornar numpy array
        pass

# Teste sua classe:
# extrator = WPTFeatureExtractor('db4', 5)
# vetor_features = extrator.extrair('senoidal_ruido.wav')
# print(vetor_features.shape) # Esperado: (32,)
```